GenotypeQuant: using microsatellites to estimate the number of different genotypes from a single species in an eDNA sample

Filipe Alberto & Lauran Liggan

2024-12-11

Background

The original motivation for this analysis was to estimate the number of kelp gametophytes (microscopic haploid stage of kelps) present in an eDNA sample using microsatellite genotyping.

Traditionally, e-DNA barcode methods estimate species presence-absence and abundance but do not estimate the number of individual genotypes present in a sample for a given target species (but see Andres et al. 2023). We devised a numerical maximum likelihood (NMLE) approach to estimate the number of contributing (NOC) genotypes most likely to have produced the observed counts of unique microsatellite alleles (OAC) amplified from e-DNA at different loci. The method is akin to forensics analysis of crime scenes with multiple suspects. Our numerical algorithm only requires the reference population allele frequencies and a vector with the OAC at each locus from the eDNA sample. We implemented a method to detect loci with lower (allele dropout) or higher (erroneously called) observed versus expected OAC and flexibility to exclude them from ML calculations. We also built in a correction for deviation from Hardy-Weinberg equilibrium in the reference populations. The NMLE has both high precision and accuracy when a relatively small number of loci are used (see Liggan et al. submitted)

Input file: microsatellite genotypes for the reference population

The input file containing the microsatellite data for the reference population is a data frame with one column per locus (no separator between the two alleles) and one row per individual. Column names are used for locus names, but there are no individual (row) codes, i.e., the first column is the information for the first locus. Use the same number of digits to code each allele. For example, 120126 is a possible value for a diploid coded with three digits per allele. At this stage, the reference data needs to be in diploid format. Missing data needs to be coded with zeros, e.g., 0, 00, 0000, or 000000.

```
# Reading a matrix with the reference population
# microsatellite alleles; one locus per column, and 3-digit
# per allele
genotypes.ref.pop <- read.delim("../Ref_Pop_Msat_All.txt")
```

The first 10 lines of the input data are shown in table 1. Note that the package does not come with an example data set but there is an example input file in the folder examples in the github page.

Table 1: The first ten lines of the genotype table for the reference population. Note that here we are using 3 digit codes for alleles.

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6
195203	193201	193199	262262	289310	280294	165165
201203	195204	193197	262272	0	280294	160165
199203	201227	199213	262262	289301	280299	162165
199199	195204	193193	260262	343347	268295	162165

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6
211221	193204	193195	272272	285293	295295	160174
196213	195201	191193	262272	0	292299	0
199199	198210	193195	272272	0	295295	160168
199215	204233	195218	266266	285351	295302	160160
195213	193193	191191	262262	285310	280295	162162
199213	204215	191195	262272	0	295295	165165

Creating OAC sampling distributions for different NOC values

The GenotypeQuant package can be installed from GitHub

```
# We need package devtools to install from GitHub
library(devtools)
install_github("UWMAlberto-Lab/GenotypeQuant")
```

We can now load the package

```
# Loading the package
library(GenotypeQuant)
```

Next, we use function *MakeAllNumberDist* to generate sampling distributions of OAC for different NOC given the reference population allele frequencies.

```
# Creating the OAC distributions for each loci and NOC value
All.Num.dist<-MakeAllNumberDist(genotypes.df=genotypes.ref.pop,ngametos=80,All.nchar=3,
                        Fis=FALSE,Fis.treshold=0.1,npermutations=5000)
```

The first argument *genotypes.df* takes the input data for the reference population (see above). The argument *ngametos* sets the maximum number of haploid genotypes g (NOC) used to sample the reference population allele frequencies. You will have different distributions from 1 to g genes for each locus, with each distribution having *npermutations* elements. When running your analysis with observed data, you can not estimate a higher number of genotypes than your *ngametos* value (sets the maximum value of the parameter space explored). For diploid genotypes, multiply *ngametos* by 2, i.e., if you are interested in estimating up to 40 genotypes, set this parameter to at least 80. Also, to visualize the maximum likelihood curve and confidence intervals *ngametos* should exceed the expected NOC. Remember, you can always rerun the function with a larger *gametes* value (generating the distributions takes a few minutes). You can use an Fis correction if your system is characterized by significant homozygosity excess (e.g., selfing). R package Genepop (Rousset 2020) will be used to calculate Qintra, the observed frequencies of identical pairs of genes in the reference population. Alleles are then sampled sequentially with probability Qintra to sample the previously sampled allele and 1-Qintra to sample any other allele according to their allelic frequencies. When applying Fis correction, it is recommended to only correct Fis values above a certain threshold. The argument *FIs.threshold* is used to set the Fis threshold and defaults to 0.1, while the *Fis* argument defaults to FALSE

Below, we show a subset for two OAC distributions for a case with seven loci. The first 20 values of the OAC distributions sampled by 5 and 15 haploid contributors are shown in Tables 2 and 3, respectively.

The function outputs a list with two components. The first slot in the results list is the 3D array with the sampling distributions of unique allele counts. The second slot contains the allele frequency table estimated from the reference population data.

Table 2: The first twenty values in the OAC distributions (columns) for seven locus (rows), sampled for 5 haploid contributors

Ner13	4	4	5	5	5	5	5	5	3	5	4	5	3	4	5	4	4	3	4	4
Ner14	3	4	5	2	4	3	3	3	4	5	4	4	4	4	3	4	3	5	4	3
Ner19	2	2	1	2	3	5	4	2	2	3	2	2	4	3	3	3	3	2	1	3
Ner2	4	2	2	3	3	2	2	2	2	2	2	3	3	1	2	4	2	2	4	2
Ner11	5	4	5	4	3	5	5	4	3	5	5	5	5	3	4	4	4	4	3	4
Ner4	4	5	3	3	3	4	3	4	5	4	5	4	3	5	5	4	3	3	4	4
Ner6	2	2	2	3	2	2	5	3	2	4	2	1	3	3	3	2	3	3	3	2

Table 3: The first twenty values in the OAC distributions (columns) for seven locus (rows), sampled for 15 haploid contributors

Ner13	7	10	9	8	7	9	9	11	10	8	10	7	9	11	12	10	9	10	8	9
Ner14	10	7	4	7	8	7	8	5	9	9	5	5	8	7	5	8	8	8	7	10
Ner19	5	5	5	4	4	5	5	6	5	4	4	5	5	4	5	4	4	4	4	5
Ner2	4	4	5	5	3	4	5	5	5	4	4	5	4	4	3	4	4	5	3	4
Ner11	11	11	10	10	8	7	10	8	8	6	9	8	10	8	8	9	9	8	11	9
Ner4	8	9	9	8	7	9	9	9	9	9	8	8	7	9	7	6	6	9	9	8
Ner6	4	4	5	3	4	3	4	7	4	4	6	3	6	4	3	4	3	5	5	3

Estimating the number of genotypes in the sample

Once the OAC distributions are created, we can estimate the NOC that most likely produced an OAC from genotyping an eDNA sample at the same microsatellite loci from the same population. At this point, the only thing we need is a vector with the OAC values for each locus.

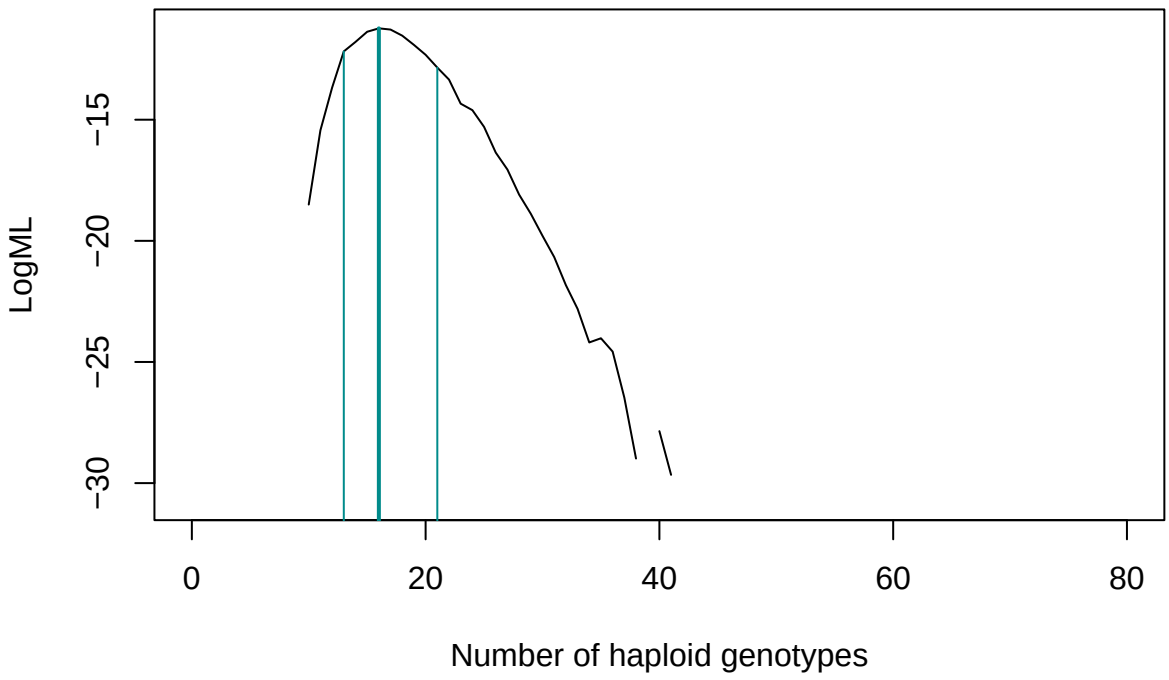
We will simulate this vector of observed unique allele counts, while you usually would be using actual eDNA genotyping data.

```
# We  simulate what an OAC created by 15 haploid genotypes could
# look like
gobs<-15
All.Numb.obs<-apply(All.Num.dist[[1]][gobs,,],1, function(x)sample(x,1))
All.Numb.obs
```

```
## [1]  9  8  6  4  8 10  3
```

Now, we can use this simulated OAC to run the algorithm. We are estimating the number of haploid genotypes here (kelp gametophytes). You could change the argument *ploidy* to 2 to estimate the number of diploid genotypes. You need to use the same loci order in this vector as in the reference population genotype table.

```
Estimate.G <- G.Estimate(Obs.N.all = All.Numb.obs, All.Num.Dist = All.Num.dist,
    ploidy = 1)
```




```
Estimate.G
```

```
## $estimateM
##      G.hat lower.95.CI upper.95.CI
## [1,]    16          13          21
##
## $LogML
##  [1]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
##  [8]      -Inf      -Inf -18.50144 -15.43915 -13.67514 -12.18002 -11.79006
## [15] -11.36930 -11.22880 -11.27912 -11.53059 -11.91179 -12.31991 -12.84846
## [22] -13.34295 -14.33823 -14.60583 -15.29548 -16.35620 -17.05932 -18.09282
## [29] -18.88271 -19.79340 -20.66807 -21.82889 -22.80839 -24.19392 -24.02144
## [36] -24.57115 -26.47157 -28.98596      -Inf -27.85489 -29.65928      -Inf
## [43]      -Inf -30.74822      -Inf      -Inf      -Inf      -Inf      -Inf
## [50]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [57]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [64]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [71]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [78]      -Inf      -Inf      -Inf
##
## $estimateM.Cor
##      G.hat.Cor lower.95.CI upper.95.CI
## [1,]        16          13          21
##
## $LogML.Cor
##  [1]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
##  [8]      -Inf      -Inf -18.50144 -15.43915 -13.67514 -12.18002 -11.79006
```

```
## [15] -11.36930 -11.22880 -11.27912 -11.53059 -11.91179 -12.31991 -12.84846
## [22] -13.34295 -14.33823 -14.60583 -15.29548 -16.35620 -17.05932 -18.09282
## [29] -18.88271 -19.79340 -20.66807 -21.82889 -22.80839 -24.19392 -24.02144
## [36] -24.57115 -26.47157 -28.98596      -Inf -27.85489 -29.65928      -Inf
## [43]      -Inf -30.74822      -Inf      -Inf      -Inf      -Inf      -Inf
## [50]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [57]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [64]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [71]      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf      -Inf
## [78]      -Inf      -Inf      -Inf
##
## $Amp.Qual
##                   Ner13    Ner14   Ner19    Ner2    Ner11     Ner4     Ner6
## Chi-square      0.22784  0.01530 1.02054 0.30351  0.24233  0.29205  0.30405
## O.minus.E      -1.55041 -0.35765 2.01632 0.96048 -1.51876  1.56914 -1.11912
## Observed.Freq   9.00000  8.00000 6.00000 4.00000  8.00000 10.00000  3.00000
## Expected.Freq  10.60000  8.40000 4.00000 3.00000  9.50000  8.40000  4.10000
##                   ChiSq  P.value
## Chi-square     2.40562 0.790636
## O.minus.E           NA       NA
## Observed.Freq       NA       NA
## Expected.Freq       NA       NA
```

The function *G.Estimate* produced a log maximum-likelihood plot with vertical lines showing the estimated number of haploid genotypes that could have generated the OAC and the respective 95% confidence interval bounds. The function also returns a list with five components. *estimate* contains the values for the estimated number of genotypes (G) and 95% CI bounds. *logML* contains the log-likelihood values for the different values of G. *estimateM.Cor* and *logML* are the corrected version of the estimator where loci with OAC equal to the count of all the alleles in the reference population at that loci are excluded from the estimation (Egeland et al., 2003; Andres et al., 2023). Finally, *AmpQual* is a table summarizing the analysis comparing the OAC at each locus with expected counts, which aims to help identify loci that might suffer from allele dropout (deficit) or alleles erroneously called (excess). Expected allele counts are calculated based on the total number of alleles amplified at all loci and the mean values of distributions of allele counts derived from the reference population (see *detecting amplification problems* below). The AmpQual table contains the following information: The first row shows the Chi-Square cell values [(0-E)ˆ2)/E] for each locus. The Chi-Square value (the sum of the values for all loci) and the p-value are the last two items in the first row. The 2nd row shows the difference between observed and expected frequencies, which indicates the direction of deviations and identifies candidate loci to exclude from ML calculations - see below). Finally, the Observed and Expected frequencies are shown in the last two rows.

Running more than one sample

Often, there will be more than one sample from which to estimate NOC. Below, we show a file with 20 samples (rows) with an OAC for each of the seven loci used. The only column that you would not have in a real data set is the last one showing the "real" G value; here, we simulated the observed data and, therefore, know the *real* number of haploid genotypes. This provides a good opportunity to evaluate the precision of the method.

```
ObservedData<-read.delim("Observed samples.txt")
```

Table 4: Example of an input file (ObservedData) with observed data for the OAC from 20 eDNA samples for the same seven loci

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6	G.real
9	8	4	5	10	9	3	17
10	10	5	5	7	9	3	15
13	9	4	4	11	9	4	22
10	10	5	4	11	11	6	21
9	10	6	4	11	11	6	20
7	4	2	5	4	7	5	10
13	10	5	4	10	12	5	24
9	7	4	5	9	9	5	17
9	8	4	4	9	8	4	21
5	5	4	4	5	4	4	7
6	5	5	3	5	5	4	6
5	5	5	3	4	5	3	7
6	4	2	3	5	5	4	7
9	9	5	4	13	8	4	17
6	4	4	2	5	5	4	8
4	2	4	2	4	3	3	4
9	10	4	4	11	7	5	15
3	3	2	3	2	2	3	4
4	3	2	3	3	3	2	4
8	7	4	5	7	5	4	12

Finally, we apply a loop to function *G.Estimate* over each of the observed samples. Then, we store the results in a new object and combine them with the observed OAC data for a final output file. Note that we turned the plotting functionality off to simplify the script. You could edit the script using pdf() to export the plots for all 20 samples to an external file.

```
# A data.frame to store the results
results.G.DF <- data.frame(matrix(nrow = 20, ncol = 3))
names(results.G.DF) <- c("G.hat", "lower.95.CI", "upper.95.CI")

for (s in 1:nrow(ObservedData)) {
    results.G.DF[s, ] <- G.Estimate(Obs.N.all = as.numeric(ObservedData[s,
        1:7]), All.Num.Dist = All.Num.dist, ploidy = 1, ML.curve = F)$estimateM
    # note the column indexation (1:7), to remove the last
    # column with the 'real' number of gametophytes
}
# Finally we bind the results with the observed data
Output <- cbind(ObservedData, results.G.DF)
```

Table 5: Binding the estimation results (last three columns) to the observed data. The first seven columns have the OAC from each locus. The G.real column shows the 'real' number of haploid genotypes (not usually known). G.hat is the NOC or the estimated number of haploid genotypes using this data. The lower and upper bounds of the 95% confidence interval are given in the last two columns.

Ner13	Ner14	Ner19	Ner2	Ner11	Ner4	Ner6	G.real	G.hat	lower.95.CI	upper.95.CI
9	8	4	5	10	9	3	17	16	13	22
10	10	5	5	7	9	3	15	16	13	22
13	9	4	4	11	9	4	22	23	17	29
10	10	5	4	11	11	6	21	23	19	32
9	10	6	4	11	11	6	20	23	19	31
7	4	2	5	4	7	5	10	8	7	10
13	10	5	4	10	12	5	24	26	21	36
9	7	4	5	9	9	5	17	16	13	21
9	8	4	4	9	8	4	21	15	12	20
5	5	4	4	5	4	4	7	6	6	8
6	5	5	3	5	5	4	6	8	6	10
5	5	5	3	4	5	3	7	6	5	8
6	4	2	3	5	5	4	7	6	6	8
9	9	5	4	13	8	4	17	19	16	27
6	4	4	2	5	5	4	8	7	6	8
4	2	4	2	4	3	3	4	4	4	5
9	10	4	4	11	7	5	15	18	14	24
3	3	2	3	2	2	3	4	3	3	3
4	3	2	3	3	3	2	4	4	4	4
8	7	4	5	7	5	4	12	11	9	14

With only seven loci genotyped and moderate allelic diversity for this reference population, the 95% confidence intervals are not too wide for ten or fewer real NOC. The precision can be improved by using more loci, more polymorphic loci, or both (see Liggan et al. submitted).

Detecting amplification problems

An eDNA sample might fail to amplify all the microsatellite alleles present (allele dropout). More concerning, for problematic samples, there might be heterogeneity across loci in the number of alleles that are amplified. This can result in OAC across loci that do not follow the different allele richness of each loci in the reference population. For example, a marker with more alleles in the reference population might have fewer amplified alleles in the eDNA sample than a loci with fewer alleles in the reference population. This deficit will result in a biased low estimate of the number of genotypes present in the eDNA sample. Conversely, a locus might have more alleles than expected due to scoring errors (e.g., stutter bands erroneously scored as alleles). This excess will result in a biased high estimate of genotypes present in the eDNA sample. We used a simple chi-square statistic to measure how the observed number of alleles amplified compares with the expected number, given the diversity of the different loci in the reference population and the total number of alleles amplified. The chi-square statistic can be used as a quality measure for the eDNA sample, the allele scoring process, or both. For each locus, the statistic ((0-E)^2)/E is given, as well as the simple difference in its numerator (O-E) to provide a measure of deficit or excess and allow selection loci that can be excluded from the estimation (see below). Also given is the sum of these values (the Chi-square statistic) and the p-value with degrees of freedom equal to the number of loci minus two. This data is not appropriate for a Chi-Square test, so the p-value should be analyzed carefully without paying too much attention to statistical significance. Still, the p-value will be smaller the larger the deviations between the observed and expected OAC.

Below, as an example, we again generate a possible vector of OAC by sampling the reference population for a likely outcome with a number of genotypes G=15 and visualize the *G.estimate* function output again. We then replace the number of alleles for the first locus, 7, by 2, a value much lower than expected for sampling this locus from the reference population for G=15. We leave the number of alleles found for other loci untouched. Note how the (0-E) for this locus increases from -0.13 to -6.37, as well as the overall chi-square statistic from 2.47 to 9.21, while the p-value decreases from 0.78 to 0.10. Also, note how the G estimate decreased from 15 to 11. Strong deficits or excesses of the observed allele counts might result in cases where an estimate can not be obtained; an NA will be returned for an estimate and CI limits. Excluding problematic loci should allow you to get an estimate (see below).

```
# We simulate what an observed sample created created by 15 haploid
# genotypes could look like
gobs <- 15
All.Numb.obs <- apply(All.Num.dist[[1]][gobs, , ], 1, function(x) sample(x,
    1))
All.Numb.obs
```

```
## [1] 10 10  5  4  6  7  4
```

```
Estimate.G <- G.Estimate(Obs.N.all = All.Numb.obs, All.Num.Dist = All.Num.dist,
    , ML.curve = FALSE, ploidy = 1)
Estimate.G$estimateM
```

```
##      G.hat lower.95.CI upper.95.CI
## [1,]    15          12          19
```

```
Estimate.G$Amp.Qual
```

```
##                   Ner13    Ner14   Ner19    Ner2    Ner11     Ner4    Ner6
## Chi-square      0.00174  0.48872 0.38586 0.39638  1.05557  0.14515 0.00062
## O.minus.E      -0.13275  1.97980 1.20939 1.07649 -3.09916 -1.08316 0.04940
## Observed.Freq 10.00000 10.00000 5.00000 4.00000  6.00000  7.00000 4.00000
## Expected.Freq 10.10000  8.00000 3.80000 2.90000  9.10000  8.10000 4.00000
##                  ChiSq  P.value
## Chi-square     2.47403 0.780401
## O.minus.E           NA       NA
## Observed.Freq       NA       NA
## Expected.Freq       NA       NA
```

```
# replacing the number of observed alleles in Loci 1 by a lower number
All.Numb.obs[1] <- 2
Estimate.G <- G.Estimate(Obs.N.all = All.Numb.obs, All.Num.Dist = All.Num.dist,
    ML.curve = FALSE, ploidy = 1)
Estimate.G$estimateM
```

```
##      G.hat lower.95.CI upper.95.CI
## [1,]    11          11          11
```

```
Estimate.G$Amp.Qual
```

```
##                   Ner13    Ner14   Ner19    Ner2    Ner11    Ner4    Ner6
## Chi-square      4.84840  1.71885 1.11509 1.04012  0.30604 0.01559 0.16619
## O.minus.E      -6.37053  3.37462 1.86863 1.58492 -1.51670 0.32260 0.73646
## Observed.Freq  2.00000 10.00000 5.00000 4.00000  6.00000 7.00000 4.00000
## Expected.Freq  8.40000  6.60000 3.10000 2.40000  7.50000 6.70000 3.30000
##                  ChiSq  P.value
## Chi-square     9.21028 0.100965
## O.minus.E           NA       NA
```

```
## Observed.Freq      NA       NA
## Expected.Freq      NA       NA
```

Upon running the function once and visualizing the results above for the first locus, we remove this locus from the estimation. This can be achieved easily by providing this locus's index to argument *exclude* in function *G.estimate*. In this example, we exclude a single locus, but several loci can be supplied using a vector of indexes. By excluding the first locus, the estimate's accuracy improved to 14 (the actual value of G is 15). Note that the table with Chi-Square statistics is not altered by using the argument *exclude*.

```
Estimate.G <- G.Estimate(Obs.N.all = All.Numb.obs, All.Num.Dist = All.Num.dist,
    ML.curve = FALSE, ploidy = 1, exclude = 1)
Estimate.G$estimateM
```

```
##      G.hat lower.95.CI upper.95.CI
## [1,]    14          11          19
```

```
Estimate.G$Amp.Qual
```

```
##                   Ner13    Ner14   Ner19    Ner2    Ner11    Ner4    Ner6
## Chi-square      4.84840  1.71885 1.11509 1.04012  0.30604 0.01559 0.16619
## O.minus.E      -6.37053  3.37462 1.86863 1.58492 -1.51670 0.32260 0.73646
## Observed.Freq   2.00000 10.00000 5.00000 4.00000  6.00000 7.00000 4.00000
## Expected.Freq   8.40000  6.60000 3.10000 2.40000  7.50000 6.70000 3.30000
##                   ChiSq  P.value
## Chi-square      9.21028 0.100965
## O.minus.E            NA       NA
## Observed.Freq        NA       NA
## Expected.Freq        NA       NA
```

References

Andres, K.J., Lodge, D.M., Sethi, S.A. and Andrés, J., (2023). Detecting and analysing intraspecific genetic variation with eDNA: From population genetics to species abundance. Molecular Ecology, 32(15), pp.4118-4132.

Egeland, T., Dalen, I., & Mostad, P. F. (2003). Estimating the number of contributors to a DNA profile. International Journal of Legal Medicine, 117(5), 271–275.

Liggan, L., Rolheiser, K.C., Pontier, O., Ramírez-Ibaceta, B., Giménez, I., Alberto, F. (submitted) Beyond presence and absence: using eDNA to estimate densities of microscopic life forms in wild populations. Molecular Ecology Resources

Rousset, F., Lopez J, Belkhir K. (2020) Package 'genepop'. R package version. Feb 23;1(7).